



Ministério da Educação  
Universidade Federal do Cariri



UNIVERSIDADE FEDERAL DO CARIRI - UFCA  
PRÓ-REITORIA DE GRADUAÇÃO - PROGRAD  
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT  
CURSO DE BACHARELADO EM ENGENHARIA DE SOFTWARE

## ESPECIFICAÇÃO TRABALHO PRÁTICO DA DISCIPLINA DE PARADIGMAS DE PROGRAMAÇÃO

### 1. Objetivo Geral

Realizar uma pesquisa aplicada sobre uma linguagem de programação, paradigma de programação ou tecnologia relacionada ao desenvolvimento de software, desenvolvendo um pequeno protótipo funcional que permita compreender seus principais conceitos, aplicações práticas, vantagens, limitações e diferenças em relação a outras tecnologias.

### 2. Objetivos de Aprendizagem

Ao final do trabalho, espera-se que os estudantes sejam capazes de:

- Aprofundar o conhecimento sobre uma linguagem de programação, paradigma ou tecnologia além dos conteúdos abordados em sala de aula;
- Compreender os problemas que motivaram o surgimento da tecnologia estudada e quais desafios ela busca resolver;
- Identificar os principais conceitos, mecanismos e características da tecnologia investigada;
- Analisar criticamente vantagens, limitações e cenários de aplicação;
- Desenvolver pequenos exemplos práticos que demonstrem os conceitos estudados;
- Comparar diferentes linguagens e paradigmas sob aspectos como desempenho, segurança, produtividade, facilidade de desenvolvimento e manutenção;
- Produzir documentação técnica organizada e bem estruturada;
- Comunicar resultados técnicos de forma clara e objetiva.



### 3. Requisitos Educacionais Obrigatórios do Projeto

Independentemente do tema escolhido (ver o item 4), todos os grupos deverão atender aos seguintes requisitos:

- Realizar uma pesquisa bibliográfica sobre a linguagem, paradigma ou tecnologia estudada;
- Apresentar uma contextualização histórica, discutindo os problemas que motivaram seu surgimento e os objetivos para os quais foi concebida;
- Descrever os principais conceitos, características, mecanismos de funcionamento e aplicações da tecnologia;
- Comparar a tecnologia estudada com pelo menos uma linguagem ou tecnologia relacionada, discutindo vantagens, limitações e cenários de aplicação;
- Desenvolver exemplos práticos, preferencialmente inspirados em problemas do mundo real, que demonstrem os principais conceitos abordados;
- Quando tecnicamente viável, apresentar exemplos de **interoperabilidade com Python** ou outras linguagens de programação;
- Realizar uma análise crítica sobre a maturidade da tecnologia, seu ecossistema, ferramentas disponíveis, comunidade, perspectivas de adoção e limitações.

Além disso, cada grupo deverá manter **um repositório público no GitHub**, que será compartilhado com toda a turma ao final do trabalho e servirá como material de consulta para estudos futuros. O repositório deverá conter, no mínimo:

- Documentação produzida majoritariamente em Markdown (.md), organizada de forma clara e estruturada, com hyperlinks, etc;
- Todo o código-fonte desenvolvido para os exemplos práticos apresentados;
- Instruções completas para instalação, configuração e execução dos exemplos, permitindo sua reprodução por outros estudantes;
- Seção com referências bibliográficas e demais materiais consultados durante a pesquisa;
- Histórico de *commits* que evidencie o desenvolvimento do trabalho ao longo do período da atividade.

A qualidade da organização do repositório e da documentação produzida será considerada como parte da avaliação do trabalho.

### 4. Tópicos de Pesquisa

#### 1. Bibliotecas de Alto Desempenho em Fortran Integradas ao Python

Bibliotecas escritas em Fortran continuam sendo amplamente utilizadas em aplicações científicas, simulações numéricas, meteorologia, engenharia e computação de alto desempenho devido à eficiência dessa linguagem em operações matriciais e cálculos intensivos. O grupo deverá investigar por que bibliotecas modernas ainda utilizam Fortran, estudar mecanismos de interoperabilidade (como f2py) e desenvolver um pequeno exemplo em que uma rotina implementada em Fortran seja utilizada por uma aplicação Python. O



trabalho deverá incluir uma comparação de desempenho e uma discussão sobre os cenários em que essa abordagem é vantajosa.

## **2. Sistema Especialista para Diagnóstico Médico em Prolog**

Apesar do crescimento do aprendizado de máquina, sistemas especialistas continuam sendo empregados em aplicações que exigem representação explícita do conhecimento e tomada de decisão baseada em regras. O grupo deverá estudar os fundamentos da programação lógica utilizando Prolog e desenvolver um pequeno sistema especialista capaz de inferir diagnósticos a partir de sintomas informados pelo usuário. Sempre que possível, a base de conhecimento deverá ser acessada por uma aplicação Python, permitindo discutir as vantagens da programação lógica em comparação com abordagens imperativas tradicionais.

## **3. Programação Concorrente com Threads e Processos em Python**

Aplicações modernas frequentemente precisam executar múltiplas tarefas simultaneamente, seja para aumentar o desempenho, melhorar a utilização dos recursos computacionais ou atender diversos usuários ao mesmo tempo. O grupo deverá investigar os modelos de concorrência disponíveis em Python (threads, processos e programação assíncrona), discutir o impacto do Global Interpreter Lock (GIL) e desenvolver experimentos comparativos utilizando cenários reais, como processamento de imagens, download simultâneo de arquivos, processamento de grandes volumes de dados ou servidores concorrentes.

## **4. Desenvolvimento de Bibliotecas em Rust Integradas ao Python**

Rust vem sendo adotada por grandes projetos de software devido à combinação entre alto desempenho e segurança de memória, eliminando diversas classes de erros comuns em C e C++. O grupo deverá estudar os mecanismos de interoperabilidade entre Rust e Python, utilizando ferramentas como PyO3 ou maturin, implementando uma biblioteca simples em Rust que possa ser utilizada em Python. O trabalho deverá discutir as vantagens dessa arquitetura em aplicações que exigem elevado desempenho sem abrir mão da produtividade oferecida pelo ecossistema Python.

## **5. Introdução à Programação Orientada a Objetos com C++**

Embora Java seja frequentemente utilizada no ensino de Programação Orientada a Objetos, C++ oferece uma implementação bastante diferente dos conceitos de orientação a objetos, conciliando recursos de programação procedural e orientação a objetos em uma mesma linguagem. O grupo deverá apresentar os principais mecanismos de encapsulamento, herança, polimorfismo, classes abstratas, sobrecarga de operadores e templates, discutindo suas diferenças em relação ao Java e ao Python. O trabalho deverá incluir exemplos práticos que demonstrem vantagens e limitações dessa abordagem em aplicações reais.

## **6. Introdução ao Rust: Segurança de Memória sem Garbage Collector**

Rust foi criada para resolver diversos problemas historicamente presentes em aplicações desenvolvidas em C e C++, especialmente aqueles relacionados à segurança de memória, concorrência e comportamento indefinido. O grupo deverá estudar os conceitos de ownership, borrowing e lifetimes, explicando como esses mecanismos substituem o



gerenciamento manual de memória sem utilizar um garbage collector. O trabalho deverá comparar pequenas implementações equivalentes em Rust e C, evidenciando quais erros são prevenidos automaticamente pelo compilador.

### **7. Julia para Computação Científica: Desempenho e Produtividade**

Julia foi desenvolvida para oferecer desempenho próximo ao de linguagens compiladas mantendo uma sintaxe de alto nível semelhante à do Python e do MATLAB. O grupo deverá investigar os principais diferenciais da linguagem, incluindo compilação Just-in-Time (JIT), múltiplo despacho (multiple dispatch) e tipagem dinâmica opcional. Também deverá ser realizada uma comparação entre implementações equivalentes em Julia e Python, discutindo quando o uso de Numba ou outras técnicas de compilação JIT em Python reduz essa diferença de desempenho.

### **8. Go: Concorrência Baseada em Goroutines e Channels**

Go foi projetada para simplificar o desenvolvimento de aplicações concorrentes e distribuídas. O grupo deverá estudar como goroutines e channels permitem implementar comunicação entre tarefas concorrentes de forma segura e eficiente, comparando essa abordagem com threads tradicionais em Python, Java ou C++. O trabalho deverá apresentar um pequeno estudo de caso envolvendo aplicações reais, como servidores Web, processamento paralelo ou sistemas distribuídos.

### **9. Zig: Uma Alternativa Moderna ao C**

Zig é uma linguagem recente que busca oferecer desempenho semelhante ao C, porém com maior segurança, ferramentas modernas e melhor gerenciamento do processo de compilação. O grupo deverá investigar seus principais objetivos, comparando recursos como tratamento de erros, gerenciamento de memória, interoperabilidade com C e sistema de build. O trabalho deverá apresentar pequenos exemplos comparativos entre Zig e C, discutindo em quais tipos de aplicações Zig vem sendo adotada.

### **10. Kotlin: Evolução da Programação Orientada a Objetos na JVM**

Kotlin tornou-se uma das principais linguagens da plataforma JVM e atualmente é amplamente utilizada no desenvolvimento Android e em aplicações backend. O grupo deverá apresentar seus principais recursos, como segurança contra referências nulas, funções de extensão, data classes e programação funcional integrada, comparando-os com Java. O trabalho deverá demonstrar como Kotlin busca reduzir a quantidade de código necessária para resolver problemas comuns da programação orientada a objetos.

### **11. Elixir e o Modelo de Atores para Sistemas Distribuídos**

Elixir utiliza a máquina virtual Erlang e foi projetada para aplicações altamente concorrentes, tolerantes a falhas e distribuídas. O grupo deverá estudar o modelo de atores, explicando como processos leves e troca de mensagens diferem das abordagens tradicionais baseadas em threads. O trabalho deverá apresentar exemplos inspirados em aplicações reais, como sistemas de mensagens instantâneas, servidores de alta disponibilidade ou plataformas de telecomunicações.



### **12. Haskell e Programação Funcional Pura**

Haskell representa uma das principais linguagens funcionais puras, baseada em imutabilidade, avaliação preguiçosa e funções de alta ordem. O grupo deverá estudar como esses conceitos diferem da programação imperativa e orientada a objetos, implementando pequenos exemplos que evidenciem composição de funções, recursão e inferência de tipos. O trabalho deverá discutir em quais domínios esse paradigma apresenta vantagens significativas.

### **13. WebAssembly: Executando Diferentes Linguagens em Ambientes Web**

WebAssembly tornou possível executar código compilado de diversas linguagens diretamente em navegadores com desempenho próximo ao nativo. O grupo deverá investigar como linguagens como C, C++, Rust ou Go podem gerar módulos WebAssembly, apresentando um pequeno exemplo de integração com aplicações Web e discutindo as vantagens dessa tecnologia para sistemas modernos.

### **14. Lua como Linguagem de Extensão para Aplicações**

Lua é amplamente utilizada como linguagem embarcada em jogos digitais, bancos de dados e aplicações que necessitam oferecer mecanismos de personalização por meio de scripts. O grupo deverá estudar como aplicações escritas em C ou C++ incorporam interpretadores Lua, implementando um pequeno exemplo que demonstre a extensão dinâmica do comportamento de uma aplicação por meio de scripts.

### **15. R: Programação para Estatística e Ciência de Dados**

Estudar a linguagem R sob a perspectiva de suas características de programação, comparando sua abordagem para manipulação de dados, estatística e visualização com Python. O trabalho deverá apresentar exemplos envolvendo análise exploratória de dados e discutir em quais cenários R continua sendo preferida por pesquisadores e estatísticos.

### **16. COBOL e a Evolução dos Sistemas Legados**

Apesar de ter sido criada há mais de seis décadas, COBOL continua sendo utilizada em milhares de sistemas críticos de instituições financeiras, seguradoras, empresas de grande porte e órgãos governamentais em todo o mundo. O grupo deverá investigar as razões que explicam a longevidade dessa linguagem, seus principais conceitos, características e paradigma de programação, bem como os desafios envolvidos na manutenção e modernização de sistemas legados. O trabalho deverá apresentar exemplos práticos desenvolvidos em COBOL, discutir estratégias utilizadas para integrar aplicações legadas com tecnologias modernas (como APIs REST, Java ou Python) e analisar em quais contextos a migração para outras linguagens é viável ou economicamente justificável.

## **5. Grupos**

O trabalho deverá ser desenvolvido em grupos de 5 a 7 estudantes. Cada grupo desenvolverá um único tema dentre aqueles disponibilizados pelo professor. Não será permitida a repetição de temas entre os grupos.



## 6. Entrega e Apresentação

Cada grupo deverá entregar um repositório público no GitHub contendo todo o material produzido durante o desenvolvimento do trabalho. A documentação deverá ser produzida majoritariamente em Markdown (.md) e organizada de forma clara e estruturada. O repositório deverá conter, no mínimo:

- README com descrição do projeto (nome da UFCA, disciplina, nome completo dos integrantes, professor, etc);
- Documentação técnica da pesquisa;
- Código-fonte dos exemplos desenvolvidos;
- Instruções para compilação, instalação e execução;
- Referências bibliográficas utilizadas.

A apresentação oral terá duração **entre 12 e 15 minutos**, seguida de até 5 minutos para perguntas. Os grupos poderão utilizar slides ou apresentar diretamente a documentação produzida e os exemplos desenvolvidos.

**Todos os integrantes deverão estar presentes e participar efetivamente da apresentação**, contribuindo **de maneira equilibrada** para a exposição do trabalho.

## 7. Avaliação

O trabalho valerá até 4,0 pontos na segunda unidade da disciplina. A nota será composta por componentes coletivos (relativos ao trabalho do grupo) e componentes individuais (relativos à participação de cada integrante na apresentação e à demonstração de domínio do conteúdo).

A avaliação considerará principalmente, mas não de maneira limitada, os seguintes aspectos:

- Qualidade e profundidade da pesquisa realizada;
- Compreensão dos conceitos fundamentais da linguagem, paradigma ou tecnologia estudada;
- Capacidade de contextualizar o problema que motivou o surgimento da tecnologia e seus principais objetivos;
- Qualidade técnica das comparações realizadas com outras linguagens ou tecnologias;
- Relevância e qualidade dos exemplos práticos apresentados;
- Capacidade de relacionar a tecnologia estudada a aplicações reais;
- Organização e qualidade da documentação produzida no GitHub;
- Clareza, objetividade e domínio do conteúdo durante a apresentação oral;
- Capacidade de responder adequadamente aos questionamentos;
- Participação individual, de maneira equilibrada e com discussões aprofundadas, de cada integrante no desenvolvimento e na apresentação do trabalho.